Persistent Local Constraint-State Patching versus Full-State Regeneration

Experiment design, fairness protocol, and CPU pilot results

Internal technical report — CPU phase

2026–08–03

Abstract

We compare two designs for adapting a robot's task state when a user interrupts a long-horizon task: CoPE (Constraint-state Patch Editing), which edits a persistent constraint state with minimal typed patches, and FSR-PC, an *internally defined* strong baseline that regenerates the complete state at every interruption. The comparison is run on a purpose-built multi-object basket-sorting benchmark whose nominal reliability is gated at $\geq 8/10$ before any method is compared on it, so that adaptation failure is not confounded with execution failure. This report documents the method, the baseline's provenance, the benchmark, the fairness controls, the metrics, and the CPU pilot. Both arms enter one shared downstream stack — the frozen online-repair engine — so that only their state-update semantics differ; every interruption is shown to drive continuation capture, runtime operator synthesis, sequential rollout verification, hard rejection, graph splicing, restore validation and stage resumption. On the CPU pilot the two arms are *tied on every behavioural outcome*; the measurable differences are in state representation, edit locality, and which audit questions the decision record can answer. No robotics claim is made: the development machine is CPU-only and no GPU software was installed, imported, or executed.

Contents

1 Scope, and what is deliberately not claimed

1. The earlier ReKep online-repair work is retained as an **integration case study only**. Its base task has a low nominal success floor and confounds execution failure with recovery failure, which makes it unsuitable as the primary benchmark for a question about task-state adaptation.
2. The existing online repair core is **reused, not rewritten**. The CoPE layer sits above it and an adapter maps between the two representations; nothing in the frozen engine was replaced or duplicated.
3. FSR-PC is **not an established published algorithm** (Section 3). It must never be cited as prior art.
4. All numbers in this report come from the **frozen 2D synthetic environment** on CPU. Rollout verification is therefore meaningful — the verifier simulates candidates under the same kinematics and control limits the executor uses — but this is still **not a robotics result**.
5. CoPE is **not claimed** to beat FSR-PC on final task success. On the pilot the two arms tie at 20/20.

2 The CoPE semantic layer

Terminology follows the collaborator's memo (Jingsu Li, *CoPE / RCSP Comprehensive Technical Memo*, v0.3, 2026–04–26): CoPE is *Constraint-state Patch Editing* and RCSP is *Reactive Constraint-State Patching*. No expansion used here is invented.

2.1 Slot schema

A constraint is a software-engineering object, not only a mathematical function:

$$c_i = (\text{id},\ \omega,\ \text{mode},\ \text{priority},\ \text{source},\ \text{validity},\ \text{restore},\ \text{lineage},\ \text{history}),$$

where the grounding ω is one field among nine.

2.2 Lifecycle: existence separated from participation

mode	exists	participates in planning	restorable
`active`	yes	yes	—
`suspended`	yes	no	yes
`overridden`	yes	no	yes
`expired`	yes (auditable)	no	**never**

`expired` is a soft delete: the record survives so that "which goal was cancelled, and why?" stays answerable. Slots are never removed.

2.3 Typed patch operators

The first-paper subset is {Insert, Suspend, Override, Inherit, Expire, Revalidate, Restore}. Two rules are pinned down by tests because violating them would still produce a passing benchmark number:

1. Revalidate is a *pure check*: it returns OK/DEGRADED/FAIL and never changes a mode.
2. Restore is a *commitment*: it is refused unless revalidation returns OK, refused outright for expired slots, and a Restore not preceded by a Revalidate is a validation violation rejected before any mutation.

2.4 Invariants

Checked before and after every patch: identity stability, lifecycle legality (`expired` terminal), no deletion, symmetric and acyclic override edges, per-slot history monotonicity, and guarded restoration. A patch failing pre-validation is not applied at all; the state is left untouched.

2.5 Two layers

CoPE semantic layer (this work)	repair execution layer (frozen, reused)
persistent state S_t, relation graph G_t, history H_t, typed patch operators	continuation capture → operator synthesis → legality → sequential rollout verification → splice → restore → resume

The adapter maps `ConstraintSlot(mode=SUSPENDED)` to a stage suspension, a `Patch` to a repair goal, and a slot's restore predicate plus lineage to a continuation. The repair engine remains the sole owner of execution, verification, and splicing.

3 FSR-PC: provenance and specification

FSR-PC is an internally defined baseline. The acronym does not appear in the collaborator's memo or in any other supplied material; a repository-wide search returned zero matches before this work began. The expansion used throughout, *Full-State Regeneration with Progress/Context*, comes only from the project brief that commissioned the baseline. Describe it as "our internally defined strong baseline". Do not attach a citation. Do not imply prior art.

At each interruption,

CoPE: $S_{t+1} = \text{apply}(P_t, S_t)$, $P_t = [\text{op}_1 \dots \text{op}_k]$, k small; FSR-PC: $S_{t+1} = \text{regenerate}(\text{context}_t)$.

Why it is strong, not a straw man. FSR-PC receives byte-identical input to CoPE at every interruption — a claim asserted by a test over all four conditions and three seeds, not by prose. The shared packet carries the full world and robot state, completed goals, pending and cancelled goals, the natural-language user request, all safety constraints, the complete task and event histories, perception, and the same compute budget and horizon. The implementation honours completed goals rather than re-planning them, carries prior lifecycle modes forward, collapses its own re-minted slots onto the canonical task vocabulary so repeated regenerations do not multiply records, and has access to the same naming helper CoPE uses. It is explicitly *not* made forgetful.

The single intended difference.

	CoPE	FSR-PC
adaptation output	typed patch over existing slots	complete new state
slot identity	carried	re-minted
per-slot edit history	continued	not inherently continued
lineage / override edges	explicit	not represented
edit footprint	touched subset	whole state
reason attached to a change	per operation	none

4 Benchmark: multi-object basket sorting

Three objects (milk, yogurt, butter) and three baskets. Nominal assignment: milk→A, yogurt→A, butter→B. `basket_C` exists as a spare destination so that a *repeated* redirection can end somewhere other than the nominal target; without it, condition I3 would be a no-op for final placement and the control arm would pass it for free. Two hard safety constraints are always present. Execution follows a fixed scripted order carried in the shared slot payload.

Reliability gate (Stage A): a precondition, not a result. Nominal task success must reach $\geq 8/10$ over ten seeds ($\geq 9/10$ preferred), measured with the NO-ADAPTATION arm on the uninterrupted task so that the number describes the task and the executor rather than any method. **If the gate fails, the task is changed or simplified; the methods are not tuned.** The CPU result is 10/10, and the gate is verified non-vacuous: a degraded executor ($p = 0.5$) fails it.

Interruption conditions.

I1 — cancellation + redirection
after milk: cancel yogurt, redirect butter B→A.

I2 — temporary unavailability
before any goal: basket B disappears, so the butter goal must be set aside and later brought back.

I3 — repeated interruption
the same goal is redirected twice (B→A, then A→C), plus a new safety constraint. Both updates land before butter is attempted, so the condition measures state survival rather than reaction latency.

I4 — changed-world restoration
as I2, but basket B is *moved* while unavailable, so restoring the suspended goal requires revalidating its grounding rather than restoring blindly.

5 Integration with the frozen repair engine

An internal audit of the v1 package found that the CoPE semantic layer was implemented but the online-repair execution core was neither delivered nor exercised: v1 tested *event → patch/regeneration → active-slot compilation → mock executor*. Every finding was verified and correct. This version integrates the engine.

What runs now.

$$C_t = (S_t, G_t, H_t), \qquad P_t = \epsilon(e_t, C_t, x_t), \qquad C_t^+ = \text{Apply}(C_t, P_t),$$
$$\kappa_t = \text{Capture}(\text{TP}, \text{stage}, x_t), \quad \Omega_t = \text{Generate}(C_t^+, x_t, \kappa_t), \quad \Delta_t^* = \arg\min_{\Delta} J(\Delta),$$

subject to LEGAL, ROLLOUTFEASIBLE and RESTOREVALID, followed by $\text{TP}^+ = \text{Splice}(\text{TP}, \Delta_t^*, \kappa_t)$ and Execute → Revalidate → Restore → Resume.

Everything from Capture onwards is the frozen engine, imported and called: `TaskProgram`, `Continuation`, `RepairPlanner`, `SynthesisGenerator`, the legality and feasibility filters, `RolloutVerifier`, `Scorer`, the splice, the restore gate, the controllers and guards, and `Synthetic2DEnv`. The repair core is byte-identical to the pre-pivot commit.

The bridge. The genuinely new component is a *constraint-state to repair-goal compiler*. It reads the state *diff* — never the policy — and maps each lifecycle transition onto the engine's own goal vocabulary: a re-grounded slot demands `aligned_to_current_target`; a suspended, cancelled or restored slot demands `restore_valid`; an inserted hard safety slot demands `safe_clearance`. Every requirement carries the slot and transition that induced it, so the trace answers *why* the robot had to act, not only *what* it did.

Verification is no longer vacuous. Against a symbolic mock, a rollout verifier has nothing to predict. Executing legs through the frozen environment gives it the same kinematics and control limits the executor uses, so a candidate that would collide, time out or fail to restore is genuinely rejected before execution. Two modelling errors were caught this way and corrected: a suspended goal was wrongly required to clear an obstruction, and an unavailable basket was wrongly modelled as a permanent obstacle. In both cases the verifier rejected every candidate — correct behaviour applied to an incorrectly stated problem.

Evidence. Over the 60-episode pilot: 105 repair invocations, 165 candidate programs synthesised at runtime, 165 sequentially rollout-verified, 105 graph splices, 105 restore validations, 105 stage resumptions, 0 safe-fallbacks, and 0 primary-arm episodes with an incomplete pipeline. A gate script prints the ten-marker sequence per interruption and exits non-zero if any is incomplete.

Both arms share the stack. A single `enter_repair_pipeline` entry point is called by both policies with the identical signature. Measured across conditions and seeds, the arms receive identical adaptation inputs *and* compile to identical repair goals, and every paired bootstrap interval on downstream work (repairs, candidates, rollouts, splices, restores, resumes) is exactly $[0, 0]$.

6 Fairness protocol

Every control is implemented in one place — the shared episode runner — so it cannot drift between arms, and each is asserted by a test.

F1 same task
the pairing key $(\text{task}, \text{condition}, \text{seed})$ deliberately excludes the method; the initial state is built before any policy object exists.

F2 same information
each arm receives one adaptation input built from method-independent facts; the serialized packet is fingerprinted, and the fingerprints are byte-identical at every event for all conditions and seeds.

F3 same executor
both arms emit the same compiled request structure from the same state store; the executor object and its random stream are shared, and the stream is keyed on the *physical action* so the arms get common random numbers. The compiled request contains no method marker.

F4 same budget, horizon, grader, metric code
the grader depends only on the condition and never inspects what a policy did.

F5 no cross-contamination
episodes are independent; store, trace, and executor are fresh per episode.

F6 one shared repair engine
a single `SharedRepairEngine` instance serves whichever policy is running, so the arms cannot diverge downstream even by accident.

One legitimate divergence, stated explicitly. The NO-ADAPTATION control can see a different *second* packet under I2/I4, because it really did something different first: it burned an action on an unavailable target. Its *first* packet is identical to the others, which is the information test. For the comparison the paper makes — CoPE vs FSR-PC — packets are identical at every event.

Statistics. Exact McNemar on discordant pairs with Wilson 95% intervals for binary outcomes; paired bootstrap (10 000 resamples, seed 0) for continuous outcomes; Holm–Bonferroni across the binary family. The statistics module is reused unchanged from the frozen CPU work.

7 Metrics

Five families, all computed by one function from the shared store, trace, and executor log: final behaviour; progress and continuity; state representation; editing versus regeneration; and audit. `None` always means *not applicable to this episode*, never *failed*.

Audit, and why it is not circular. The record must answer: Q1 which goal was cancelled; Q2 which completed goal was preserved; Q3 why was a constraint suspended; Q4 what condition allowed restoration; Q5 which state replaced the old target. A trace made only of CoPE operations would answer all five by construction and score FSR-PC at zero for circular reasons. Every query therefore has *two* evidence paths: typed patch operations, *and* diffing consecutive regenerated snapshots. Q1 and Q2 are recoverable from a state snapshot, and FSR-PC answers them. Q3–Q5 ask for a reason, a restoration condition, and a replacement identity — none of which a complete snapshot carries. That asymmetry is the finding. Queries a condition does not pose score `None`, never `False`.

8 CPU pilot results

Stage A: 10/10 → PASS. Stage B: 60 episodes (5 paired seeds × 4 conditions × 3 arms).

Stage C ablation: mechanism, not rhetoric.

arm	success	identity preserved	mean edit distance	mean audit coverage
CoPE	20/20	20/20	0.507	1.000
FSR-PC	20/20	0/20	1.000	0.458
FSR-PC(preserve ids)	20/20	**20/20**	1.000	0.458

Table 1: Binary outcomes, exact McNemar on 20 paired episodes per arm.

metric	CoPE	FSR-PC	p
revised task success	20/20	20/20	1.000
completed progress preserved	20/20	20/20	1.000
remaining goal correct	20/20	20/20	1.000
identity preserved	**20/20**	**0/20**	1.9×10^{-6}
lifecycle transitions legal	20/20	20/20	1.000
lineage correct	20/20	20/20	1.000
history monotonic	20/20	20/20	1.000

Table 2: Continuous outcomes, paired bootstrap 95% CI on CoPE − FSR-PC.

metric	CoPE	FSR-PC	95% CI	excludes 0
normalized edit distance	0.457	1.000	$[-0.611, -0.472]$	yes
slots touched per event	2.75	5.13	$[-2.850, -1.875]$	yes
audit coverage	**1.000**	**0.458**	$[+0.483, +0.600]$	yes
completion steps	44.5	44.5	$[0, 0]$	no
invalid actions	0.000	0.000	$[0, 0]$	no
repairs / candidates / splices / resumes	identical		$[0, 0]$	no

Re-using identifiers recovers the identity metric and changes nothing else. The locality and audit differences therefore come from local typed editing, not from identifier hygiene.

9 Honest interpretation

Supported. The arms tie on every behavioural outcome. The measurable differences are structural: identity survival, edit locality, and which audit questions the record can answer. The audit gap is the substantive one, and the two-evidence-path scoring is what makes it non-circular.

Not supported. A normalized edit distance of 1.000 for FSR-PC is true *by definition* of full regeneration; it describes the two designs and must not be presented as an experimental finding. No physical, robotics, or wall-clock-cost claim follows from a CPU mock. Neither arm uses a language model, so no token or prompt-cost claim is made. With 20 paired episodes per arm, the behavioural ties mean *no evidence of a difference*, not *evidence of no difference*.

What would change the conclusion. The mock executor never punishes a slightly-wrong-but-recoverable state. On the GPU host, the arms may separate under longer horizons where regenerated state drifts across many interruptions; under more than two interruptions, where re-minted identity obscures what has already been attempted; or where the executor cannot cheaply retry, so a duplicated or stale goal becomes unrecoverable. If none of these separate the arms, the honest paper claim is about representation, auditability, and edit locality — not task success — and the results section should say exactly that.

10 Status and next steps

Local validation complete: 229 tests pass (94 pre-existing, unchanged; 135 new), Stage A/B/C pilots run, five side-by-side dry-run traces produced, this report compiles, and checksums are

Table 3: Per-condition final success on the physical stack. Every interruption condition now punishes not adapting.

condition	CoPE	FSR-PC	NO-ADAPTATION
I1	5/5	5/5	**0/5**
I2	5/5	5/5	**0/5**
I3	5/5	5/5	**0/5**
I4	5/5	5/5	**0/5**

recorded. Nothing GPU-related has been executed; every GPU command in the runbook is marked REMOTE GPU SERVER ONLY and is untested by construction.

The single integration point for the GPU host is the executor class. Both policies already emit the same compiled request; no policy code changes.

Authorship and attribution

The abstract idea of online constraint-program repair, and the CoPE/RCSP constraint-state formulation with its slot schema, lifecycle modes, and typed patch operators, originate with the collaborator (Jingsu Li, memo v0.3, 2026–04–26). The work documented here is the experimental realisation: the adapter onto the existing repair engine, the internally defined FSR-PC baseline, the benchmark, the fairness protocol, the metrics, and the CPU pilot. Author order and final framing are not settled here and should not be inferred from this document.